MODULE 10 — Production Interview Scenarios

Senior SWE Interview Track — Operating Systems

These are the "you're on call, the pager fires" questions that senior interviews at Google/Meta/Amazon/Uber/Netflix use to separate practitioners from textbook readers. Each scenario follows the same skeleton: **triage commands → hypothesis tree → root causes → fixes → prevention**. Interview scoring rewards *ordered, evidence-driven* investigation — always say what you'd measure before what you'd change.

The universal first minute (USE-style sweep)

```
uptime; dmesg -T | tail            # load, recent kernel events (OOM! I/O errors!)
top / htop                          # who eats CPU/mem; load vs cores; %us/%sy/%wa
vmstat 1                            # r (runnable), b (blocked), cs, si/so (SWAP!)
iostat -x 1 / sar -B                # disk util/latency, page faults
ss -s; ls /proc/<pid>/fd | wc -l    # sockets, fds
```

10.1 High CPU Usage

Why Interviewers Ask This

The most common incident; they watch whether you split user vs system vs steal time before touching code.

Triage Path

1. `top`: which process; then `top -H -p <pid>`: which **threads**.
2. Split the CPU: `%us` (your code) / `%sy` (kernel — syscalls, spinning in futex?) / `%wa` (not CPU! I/O wait) / `%si` (softirq — network) / `%st` (steal — noisy VM neighbor or cgroup throttle).
3. Profile: `perf top` / `perf record -g` (C/C++/Go), async-profiler/JFR (JVM), py-spy (Python). Flame graph the hot stacks.
4. Container check: `throttled_time` in cpu.stat — CPU *limit throttling* masquerades as slowness without high host CPU.

Root-Cause Menu (map to modules)

- Busy loop / accidental polling (M8): thread at 100% with trivial stack.
- Lock contention burning `%sy` in futex or spinning (M3).
- GC storms (allocation rate — M5); regex/serialization hot spots.
- Too many runnable threads → context-switch overhead (`vmstat` cs, `pidstat -w`) (M1/M2).
- Interrupt/softirq saturation on CPU0 (M8.4).
- **Livelock**: 100% CPU, zero goodput, retry counters spiking (M4.7).

ASCII Decision Sketch

```
CPU high ->
  %st high -> neighbor/quota (move host; fix limits)
  %si high -> IRQ/softirq (affinity, RSS, coalescing)
  %sy high -> syscalls/locks (strace -c, perf; futex? spinlock?)
  %us high -> perf/flame graph -> hot function -> fix code
  none high but app slow in container -> cgroup throttling (cpu.stat)
```

Interview Traps

- Load average ≠ CPU usage (includes D-state — M1.6).
- 100% of one core on an 8-core box = 12.5% total — single-thread bottleneck, different fix (parallelize) than global saturation (scale/optimize).
- CPU high *and* healthy could be fine — tie to SLO impact before "fixing".

Practice

"API p99 doubled; host CPU 85% (was 40%) after a deploy. Walk me through to the exact function." (diff deploys, flame graph before/after, find the new hot frame, e.g., accidental O(n²) or debug logging in a loop)

10.2 High Memory Usage

Why Interviewers Ask This

Tests RSS/VSZ/cache literacy (M5) — most candidates misread `free` and "fix" non-problems.

Triage Path

1. `free -h`: is it *really* used, or page cache? (**available** is the number that matters; cache is reclaimable — M7.1.)
2. `top` sorted by %MEM / `ps aux --sort=-rss`; `smem`/PSS if shared memory muddies it (M5.1).
3. Per-process anatomy: `/proc/<pid>/status` (VmRSS breakdown), `pmap -x` — heap? mmaps? thread stacks? / dev/shm (M9.9)?
4. Container: cgroup `memory.current` vs limit; page cache counts *inside* the cgroup → "OOM" with cache-heavy workloads.
5. Trend: is it a step (new workload) or a slope (leak → 10.3)?

Root-Cause Menu

- Page cache (not a problem — don't `drop_caches` as a "fix" in interviews!).
- Legit working-set growth (traffic, bigger heap) → capacity.
- Fragmentation / freed-not-returned allocator memory (M5.9).
- tmpfs//dev/shm data counting as RAM (M9.9); huge thread counts × stacks (M1.3).
- Leaks (next scenario). Swap thrash: `vmstat` si/so > 0 sustained = the real emergency (M6.6).

Interview Traps

- "Linux ate my RAM" (cache) — say it before they bait you.
- OOM killer forensics: `dmesg` shows the score/victim; killed process ≠ guilty process (biggest ≠ leaker).
- Sum of per-process RSS > physical RAM is normal (shared pages).

Practice

"Host shows 95% memory used; nothing looks wrong. Is there a problem?" (walk free/available, cache, pressure via `sar -B`/PSI `some avg10` in /proc/pressure/memory — modern answer)

10.3 Memory Leaks

Why Interviewers Ask This

Multi-layer debugging skill: language runtime → allocator → OS, with different tools per layer.

Triage Path

1. Confirm: RSS slope over days (metrics), not a one-step jump. Restarts "fix" it → leak-shaped.
2. Identify the layer: - **Managed heap** (JVM/Go/Python): heap profiler/dump — growing object graph? (jmap+MAT, pprof heap, tracemalloc). Watch for *logical* leaks: unbounded caches, listener registries, ThreadLocals in pools. - **Native/allocator**: heap flat, RSS grows → native leak (JNI, cgo, C libs) or fragmentation (M5.9): jemalloc profiling / valgrind massif / heaptrack in staging. - **Not memory at all**: FDs/mmaps/threads also inflate — check `/proc/<pid>/fd`, `pmap` diff, thread count.
3. Kernel-side: slab leaks (`slabtop`) if system memory vanishes with no process owning it.

ASCII Decision Sketch

```
RSS climbs forever:
  managed-heap climbs too -> object leak -> heap dump diff (2 snapshots!)
  managed-heap flat       -> native: pmap diff / jemalloc prof / valgrind
  heap tools clean        -> fragmentation (allocator stats) or
                             mmap/fd/thread growth
Golden technique: TWO snapshots, diff, biggest delta wins.
```

Interview Traps

- GC exists ≠ no leaks (reachable-but-useless is the definition of a managed leak).
- Fragmentation vs leak (M5.9) — allocator stats decide.
- The “leak” that’s a cache with no eviction (M6!) — bounded caches are the fix.

Practice

“Java service OOMs every ~4 days. Design the investigation end-to-end and name the tool at each step.” (GC logs → heap dump on OOM flag → MAT dominator tree → e.g., a static Map of sessions never evicted)

10.4 Thread Starvation

Why Interviewers Ask This

The classic “service is slow but CPU is idle” paradox — tests thread-pool math (M1/M2) and dump-reading (M1.7).

Triage Path

1. Symptom check: latency up, CPU low, queue depths up.
2. Thread dumps ×3, 10 s apart (jstack / Go SIGQUIT / py-spy dump): histogram the states.
3. Patterns: - All workers **WAITING on the same external call** (DB pool empty, slow downstream) → pool exhaustion cascade. - Workers BLOCKED on one lock → contention (10.6) not starvation. - Pool threads waiting on results of tasks queued in *the same pool* → **thread-pool deadlock/starvation** (M4.1 practice) — the single most-loved interview variant (async code doing blocking `.get()` on the common pool). - Low-priority/fairness starvation (M2.8, M4.8): one class of work never scheduled.

Fix Ladder

Separate pools per dependency (bulkheads); size pools = cores × (1 + wait/compute); timeouts + circuit breakers on all blocking calls; never block inside async executors; fairness modes for starved lock waiters (Go mutex starvation mode — M4.8).

Interview Traps

- “Add more threads” without the math — more threads with a saturated downstream just queues harm (and adds switch overhead — M1.5).
- Starvation vs deadlock vs slow dependency: dumps distinguish them; say how.

Practice

“Tomcat: 200/200 threads busy, CPU 10%, DB pool 20 connections at 100% utilization, DB p99 2 s. Explain the cascade and fix at every layer.” (Little’s law on the pool, timeouts, pool sizing, DB query fix)

10.5 Deadlocks (in production)

Why Interviewers Ask This

M4 theory turned operational: can you *find* one at 3 a.m.?

Triage Path

1. Smell: a subset of requests hang forever (no timeout errors — just stuck); thread count stable; specific endpoints frozen.
2. In-process: thread dump — JVM literally prints "Found one Java-level deadlock" with the cycle; Go: goroutine dump shows mutual channel/mutex waits; C++: gdb, inspect lock owners.
3. Database: `SHOW ENGINE INNODB STATUS` (latest deadlock), pg_locks joined with pg_stat_activity; error 1213/40P01 rates in app logs.
4. Distributed: no global detector — look for lock-timeout logs, stuck sagas, two services each holding a row the other wants.

Fix Ladder

Immediate: restart (in-process) / kill victim TX (DB). Real fix: lock ordering (M4.2), shrink transactions, single-lock-then-call-out design, jittered retry on DB deadlock errors (M4.6). Prevention: lock-order assertions in debug builds, CI deadlock tests with contention harnesses, alert on deadlock counters.

Interview Traps

- Distinguish "deadlocked" from "waiting on a dead downstream forever" (missing timeout) — dumps show *what* it waits on.
- DB deadlocks are *normal at low rate*; the bug is missing retries or hot-row design (say both).
- jstack shows the cycle but you still must map monitors → code sites → ordering fix.

Practice

"Every few days, 3 endpoints freeze until restart; dumps attached showing T1 holds A wants B, T2 holds B wants A across two @Service classes. Give the immediate action, the code fix, and the CI guard."

10.6 Lock Contention

Why Interviewers Ask This

The subtler cousin of deadlock: everything *works*, just slowly — tests M3 cost models and scalability instincts.

Triage Path

1. Signature: throughput plateaus (or *drops*) as load/cores rise; CPU has headroom; p99 >> p50.
2. Measure, don't guess: JFR/async-profiler lock profiles (which monitor, how long), `perf lock record`, Go mutex profile (`runtime.SetMutexProfileFraction`), `%sy` +futex time in perf.
3. Identify the hot lock and its **hold time × acquisition rate** (utilization of the lock — M3.2's queueing math).

Fix Ladder (in order of preference)

1. **Shrink the critical section** (move I/O/alloc/logging out — M3.2).
2. **Shard/stripe** the lock (per-bucket locks, `ConcurrentHashMap`-style; per-core counters for stats — M3.9).
3. Change structure: read-mostly → RWLock or better snapshot-swap/RCU (M3.6); queues → lock-free/MPSC (M3.10).
4. Reduce arrival rate: batch under one acquisition; thread-local buffers flushed periodically.

Interview Traps

- RW lock as a reflex "fix" can be slower (M3.6); contended atomic counters still serialize (M3.9 false sharing/ping-pong).
- Adding threads to a contended system *reduces* throughput (convoying, switch overhead) — say the negative-scaling curve out loud.
- Amdahl: 5% serialized caps you at 20× — quantify before micro-optimizing.

Practice

"A metrics library's global histogram mutex is 40% of profile samples at peak. Design the fix (striped/per-CPU + periodic merge) and estimate the improvement."

10.7 Too Many Open Files

Why Interviewers Ask This

Beloved because it chains config (ulimits), code (leaks), and protocol (CLOSE_WAIT) knowledge — M7.6 + M9.8 in one incident.

Triage Path

1. Error: `EMFILE` (per-process limit) vs `ENFILE` (system `fs.file-max`) — different knobs.
2. Count and classify: `ls /proc/<pid>/fd | wc -l`; `lsof -p <pid>` → sockets? files? pipes? anon inodes (epoll/eventfd)?
3. Sockets dominating: `ss -tan | awk '{print $1}' | sort | uniq -c`: - **CLOSE_WAIT** pile = *your code* never closes after peer FIN → FD leak, fix close paths. - **TIME_WAIT** pile = churny short connections (usually fine; ports, not FDs — but signals missing keep-alive/pooling).
4. Check the limit story: soft vs hard `ulimit -n`, systemd `LimitNOFILE`, container defaults — 1024 default vs 10k+ connection reality.

Fix Ladder

Immediate: raise soft limit (buy time), restart the leaker. Real: find the leak (error paths missing close — use RAII/defer/try-with-resources), add HTTP client pooling/keep-alive, close on all exception branches. Prevention: alert at 70% of `fd_used/fd_limit`; load tests that assert FD count returns to baseline.

Interview Traps

- Raising the limit is mitigation, not the fix — say it explicitly.
- Everything is an FD: epoll instances, timerfd, inotify watches count too (M7.6).
- CLOSE_WAIT vs TIME_WAIT direction (who owes the close) is a favorite rapid-fire check.

Practice

"Proxy dies nightly with EMFILE at ~1024 FDs; lsof shows 900 CLOSE_WAIT to one upstream. Reconstruct the bug (upstream closes idle conns; proxy ignores EOF) and fix at code + config + alerting levels."

10.8 Process Crashes

Why Interviewers Ask This

Postmortem methodology under pressure: signals, cores, OOM forensics — M9.6 + M5 applied.

Triage Path

1. **How did it die?** Exit code decodes first: 128+N = killed by signal N (137 = SIGKILL → almost always **OOM killer or K8s limit**; 139 = SIGSEGV; 134 = SIGABRT/assert; 143 = SIGTERM — someone asked politely).
2. External evidence: `dmesg -T | grep -i oom` (killer logs victim + scores), `journalctl -u svc`, K8s `OOMKilled` status/events.
3. SIGSEGV path: enable core dumps (`ulimit -c`, `kernel.core_pattern`, coredumpctl) → `gdb bin core` → `bt` → faulting address analysis (NULL? stack guard = overflow (M5.10)? use-after-free — ASan in staging).
4. Crash *loop* vs one-off: supervisor backoff, crash on specific input (poison message replay!), or resource-based (OOM every N hours = leak → 10.3).

ASCII Decision Sketch

```
exit code:
 137 (SIGKILL) -> dmesg OOM? cgroup limit? someone's kill -9?
 139 (SIGSEGV) -> core dump -> gdb bt -> null/UAF/stack overflow
 134 (SIGABRT) -> assert/fatal exception/allocator abort -> logs
 143 (SIGTERM) -> orchestrator/deploy killed it (probe failures?)
 0/1..n        -> app-level exit -> logs
K8s: OOMKilled | CrashLoopBackOff (check probes!) | Evicted (node pressure)
```

Interview Traps

- OOM-killed ≠ has a leak (limit too low, cache growth, COW spike at fork — M5.8 Redis story).
- K8s liveness probe misconfig masquerades as “crashes” (SIGTERM from kubelet).
- No core dumps configured = flying blind — mention enabling them *before* the next crash.

Practice

“Container restarts every ~6 h, exit code 137, no app error logs. Full investigation, and three different root-cause families you must rule out (leak, limit-vs-working-set, COW/fork spike).”

10.9 Bonus rapid-fire scenarios (one-liners you should be able to expand)

- **Load average 60, CPU idle** → D-state pileup on NFS/disk (M1.6): `ps ... | awk '$8~/D/'`, `/proc/<pid>/stack`, fix storage.
- **Server slow after backup job** → page-cache eviction by sequential scan (M7.8): fadvise/O_DIRECT/ionice the backup.
- **p99 spikes every 30 s, exactly** → dirty-page writeback flushes / journal commits (M7.3/7.8): tune vm.dirty_*, split WAL device.
- **First request after deploy slow** → demand paging cold start (M5.7): warmup, pre-touch, mlock.
- **Service can’t be killed** → D-state (M1.6) or zombie confusion (M9.4): identify state before reaching for -9.
- **Sudden EADDRINUSE on restart** → no SO_REUSEADDR + TIME_WAIT (M9.8), or leaked child holding the listener (M9.2 CLOEXEC).
- **Box slows, si/so nonzero, disk busy** → swap thrash (M6.6): find the memory hog; do NOT add swap.
- **One CPU 100% softirq** → IRQ affinity/RSS (M8.4).

Module 10 Cheat Sheet (one page)

Exit codes: 137=SIGKILL(OOM?) · 139=SIGSEGV · 134=SIGABRT · 143=SIGTERM. **dmesg first** for OOM/hardware. **State reading**: R-pileup=CPU · D-pileup=disk/NFS (load lies!) · Z=unreaped · CLOSE_WAIT=you leak · TIME_WAIT=churn. **CPU split**: us=code · sy=kernel/locks · wa=disk(not CPU) · si=network · st=neighbor/quota. **Memory truths**: available>free matters · cache is reclaimable · RSS≠VSZ · PSI /proc/pressure/ *is the modern signal. Tool belt*: top/-H, vmstat 1, pidstat -w, iostat -x, sar -B, ss -s/-tan, lsof, /proc/pid/{fd,status,stack}, perf record -g/flamegraphs, strace -c, jstack/pprof/py-spy, dmesg, PSI.

Symptom	First command	Likely module
Slow + CPU idle	thread dumps ×3	M1.7/M3 (waits/locks)
Slow + CPU busy	perf/flame graph	M2/M3 (code/locks)
Load high CPU idle	ps state D scan	M1.6 (I/O)
RSS slope	2-snapshot heap diff	M5 (leak/frag)
Throughput drops w/ more threads	lock profile	M3 (contention)
Frozen subset of endpoints	dump → cycle	M4 (deadlock)
EMFILE	/proc/fd count + ss states	M7.6/M9.8
Restart loop 137	dmesg + cgroup limits	M5/M10.8

Top Interview Questions

1. “Service is slow” — full structured triage (they grade the *order*).
2. Load 80, CPU 10% — explain and fix.
3. OOM-killed nightly — investigate without guessing.
4. All 200 worker threads busy, CPU idle — the pool-exhaustion cascade.
5. Throughput fell when we doubled threads — negative scaling story.
6. EMFILE at midnight — CLOSE_WAIT forensics.
7. Exit code 137 vs 139 vs 143 — what each tells you.

Common Mistakes (module-wide)

- Jumping to fixes before evidence; restarting away the diagnosis (say: capture dumps/cores *then* restart).
- Misreading load average, free memory, and %wa.
- “Add threads/RAM/swap” as reflexes (often each makes it worse).
- Not knowing the two-snapshot diff technique for leaks.
- Forgetting cgroup/container limits behave differently from host limits.

Mock Interview (self-test, ~30 min — the final boss)

1. Pager: “checkout p99 5 s (was 200 ms), started 14:02.” You have one SSH session. Narrate your first 10 commands and what each rules in/out.
2. `vmstat 1` shows: r=1, b=14, si/so=0, wa=60%. Three hypotheses, one discriminating test each.
3. A Go service: CPU 100%, goroutine dump shows thousands in `runtime.gopark` on one channel and one goroutine in a tight `select` retry loop. Deadlock, livelock, or starvation? Fix?
4. K8s pod: OOMKilled at 2 GB limit; jmap heap = 800 MB steady. Enumerate where the other 1.2 GB can live (metaspace, threads×stacks, direct buffers, mmap, glibc arenas, page cache in cgroup) and how to measure each.
5. Postmortem: write the 5-line timeline + root cause + 3 preventions for: FD leak (CLOSE_WAIT) → EMFILE → accept() fails → LB marks node down → cascade. What single alert would have caught it a week earlier?